

Synthèse complète - Modélisation et Programmation Orientée Objet

L2 Informatique - Java - Semestre 3

Synthèse de révision basée sur le cours de Mohamed Morchid

Source principale : support de cours L2 MPOO Java, Mohamed Morchid, 419 pages.

Comment utiliser cette synthèse

Cette synthèse reprend l'ensemble des notions importantes du cours de **Modélisation et Programmation Orientée Objet en Java** et suit l'organisation du support original de 419 pages. Le cours est structuré en sept grands blocs : principes de base de l'OO et Java, classes et objets, héritage et polymorphisme, interfaces et lambda, packages et exceptions, collections et entrées-sorties, puis sérialisation, JDBC, threads et sockets.

L'objectif n'est pas de recopier les diapositives, mais de transformer leur contenu en un cours de révision cohérent : définitions, liens entre notions, règles Java, exemples, pièges classiques et points à savoir expliquer à l'examen. Les exemples Java ajoutés servent à illustrer les notions du cours.

Conseil de révision : pour chaque notion, essaie de pouvoir répondre à trois questions : *qu'est-ce que c'est ?*, *à quoi cela sert ?*, *comment cela s'écrit en Java ?*

Sommaire

1. Fondements de l'orienté objet et introduction à Java - pages 3 à 86 du cours
2. Classes, objets et classes utilitaires - pages 87 à 129
3. Héritage, généricité et polymorphisme - pages 130 à 207
4. Interfaces et expressions lambda - pages 208 à 275
5. Packages et exceptions - pages 276 à 341
6. Collections et entrées-sorties - pages 342 à 376
7. Sérialisation, JDBC, threads et sockets - pages 377 à 419
8. Fiche de révision finale : différences, pièges et questions types

1. Fondements de l'orienté objet et introduction à Java

Cours : pages 3 à 86.

1.1 Programmation procédurale et approche objet

En programmation procédurale, on part généralement du problème global et on le décompose en sous-problèmes plus simples. C'est une approche dite **descendante**. On distingue les **données** et les **traitements** : les données représentent l'état du programme et les fonctions réalisent des opérations sur ces données.

Un programme procédural peut donc être vu comme un assemblage de structures de données et d'algorithmes. Cette approche fonctionne bien, mais elle devient difficile à maintenir lorsque les différentes parties du programme partagent beaucoup d'informations : une modification dans une structure ou dans une fonction peut avoir des conséquences inattendues ailleurs.

L'approche objet cherche au contraire à rapprocher les données et les opérations qui les concernent. On construit le logiciel comme un assemblage de composants relativement indépendants. Le but est notamment de :

- diminuer le couplage entre les parties du programme ;
- éviter le partage inutile d'informations ;
- favoriser la réutilisation de composants ;
- faciliter l'évolution et la maintenance du logiciel ;
- modéliser les concepts du domaine de manière plus naturelle.

Exemple conceptuel

Dans une application bancaire procédurale, on pourrait avoir une structure `Compte` et des fonctions `deposer(compte, montant)` et `retirer(compte, montant)`. En programmation objet, on cherche plutôt à obtenir un objet `Compte` qui possède son propre état et expose directement les opérations autorisées :

```
Compte c = new Compte(1000.0);
c.deposer(200.0);
c.retirer(50.0);
```

L'objet regroupe ainsi les données et les opérations qui ont du sens pour lui.

1.2 Les quatre caractéristiques essentielles de l'OO

Abstraction

L'**abstraction** consiste à ne retenir que les caractéristiques essentielles d'un concept. Une classe décrit ce qui est utile au programme sans représenter tous les détails de la réalité.

Exemple : pour une voiture, un programme de location peut retenir la marque, l'immatriculation, le kilométrage et l'état de disponibilité, mais ignorer la couleur des sièges si cette information n'est jamais utilisée.

Encapsulation

L'**encapsulation** consiste à masquer l'état interne et les détails de fonctionnement d'un objet. L'utilisateur de l'objet voit principalement son **interface**, c'est-à-dire les services qu'il peut demander, mais pas nécessairement la façon dont ces services sont réalisés.

On sépare donc :

- l'**interface** : ce que l'objet permet de faire ;
- l'**implémentation** : la manière dont il réalise ces opérations.

En Java, l'encapsulation repose fortement sur les niveaux de visibilité tels que **private**, **protected** et **public**.

Modularité

La **modularité** consiste à découper le programme en modules aussi indépendants que possible. Une bonne modularité facilite le test, la maintenance et la réutilisation.

Hiérarchie

La **hiérarchie** organise les abstractions selon des relations de généralisation et de spécialisation. Dans une hiérarchie d'héritage, on exprime souvent la relation “**est une sorte de**” : un Chien est une sorte d'Animal.

1.3 Points complémentaires : typage, simultanéeité, persistance

Typage

Un type définit les valeurs possibles et les opérations autorisées sur une donnée. En Java, les classes définissent également des types.

Le cours distingue notamment :

- typage faible ou fort ;
- typage statique ou dynamique ;
- conversion explicite avec un cast ;
- compatibilité de types liée à l'héritage.

Java est présenté dans le cours comme un langage fortement typé. Le compilateur vérifie de nombreuses incompatibilités avant l'exécution.

Simultanéeité

Plusieurs objets ou traitements peuvent être actifs en même temps. Ce principe apparaîtra plus tard avec les **threads**.

Persistance

Une donnée locale ne vit habituellement que pendant une partie de l'exécution. La persistance désigne la capacité à conserver un objet ou son état au-delà de l'exécution du programme, par exemple dans un fichier ou une base de données. Le cours reviendra sur cette idée avec la sérialisation et JDBC.

1.4 Objet : état, comportement et identité

Un **objet** est une entité identifiable possédant des caractéristiques bien définies. Le cours considère les termes *objet* et *instance* comme synonymes.

Un objet possède trois dimensions essentielles :

1. **Un état** : les valeurs de ses attributs à un instant donné.
2. **Un comportement** : les opérations ou services qu'il peut exécuter.
3. **Une identité** : ce qui permet de distinguer cet objet de tous les autres, même si deux objets ont exactement les mêmes valeurs.

Exemple

Pour une voiture :

Etat : vitesse = 0, couleur = grise, carburant = 30 L

Comportement: demarrer(), accelerer(), freiner()

Identite : cette voiture précise, distincte d'une autre voiture identique

En Java, l'état est principalement représenté par les **attributs** et le comportement par les **méthodes**.

1.5 Classe et instance

Une **classe** est une abstraction décrivant un ensemble d'objets ayant une structure et des comportements communs. Elle joue le rôle de modèle ou de moule.

Une classe définit donc :

- des attributs communs à la structure des objets ;
- des opérations disponibles pour les objets de ce type.

Exemple

```
public class Voiture {  
    private String marque;  
    private int vitesse;  
  
    public void accelerer() {  
        vitesse++;  
    }  
}
```

Chaque instance de Voiture possède son propre état :

```
Voiture v1 = new Voiture();  
Voiture v2 = new Voiture();
```

v1 et v2 sont deux objets distincts de la même classe.

1.6 Instanciation et classes abstraites

L'**instanciation** est la relation entre une classe et l'objet créé à partir de cette classe. En Java, elle se fait généralement avec `new`.

```
Point p = new Point();
```

Certaines classes servent uniquement à représenter un concept général et ne doivent pas être instanciées. On parle de **classes abstraites**. Elles seront étudiées plus précisément avec l'héritage.

1.7 Encapsulation et visibilité

L'encapsulation protège l'intégrité de l'objet. Au lieu d'autoriser n'importe quelle modification directe, la classe contrôle les opérations réalisées sur ses données.

```

public class Compte {
    private double solde;

    public void déposer(double montant) {
        if (montant > 0) {
            solde += montant;
        }
    }

    public double getSolde() {
        return solde;
    }
}

```

Le champ `solde` est privé : du code extérieur ne peut pas écrire directement une valeur incohérente. Les méthodes publiques constituent l'interface de la classe.

L'intérêt majeur est que l'implémentation peut évoluer sans changer l'utilisation extérieure. Tant que la signature d'une méthode reste la même, son fonctionnement interne peut être modifié sans imposer de changement aux utilisateurs de la classe.

1.8 Construction, destruction et ramasse-miettes

Avant d'utiliser un objet, il faut le créer et initialiser son état. En Java, l'initialisation est réalisée par un **constructeur**.

```

public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

Quand un objet n'est plus accessible, Java peut récupérer automatiquement sa mémoire avec le **garbage collector** ou ramasse-miettes. Le programmeur n'effectue pas lui-même une libération de mémoire comparable à `free` en C.

1.9 Messages, opérations et méthodes

Dans la modélisation objet, les objets communiquent par **messages**. En Java, envoyer un message à un objet revient généralement à appeler l'une de ses méthodes.

```

p.deplacer(3, 4);

```

Le cours distingue :

- une **opération** dans le modèle : le service attendu ;
- une **méthode** en Java : l'implémentation de cette opération ;
- un **message** : la demande d'exécution de l'opération par un objet.

Un message peut être synchrone ou asynchrone. Dans le cas synchrone, l'appelant attend la fin de l'opération. Le cours se concentre principalement sur les appels synchrones classiques.

1.10 Membres d'instance et membres de classe

Un **membre d'instance** appartient à chaque objet. Chaque instance possède sa propre copie d'un attribut d'instance.

Un **membre de classe** est commun à toutes les instances. En Java, on utilise le mot-clé `static`.

```
public class Joueur {
    private String nom;           // attribut d'instance
    private static int nombre = 0; // attribut de classe

    public Joueur(String nom) {
        this.nom = nom;
        nombre++;
    }

    public static int getNombre() {
        return nombre;
    }
}
```

Utilisation :

```
int n = Joueur.getNombre();
```

Une méthode `static` appartient à la classe. Elle ne dispose pas directement d'un objet courant `this` et ne peut donc pas accéder directement à un attribut d'instance sans disposer d'une référence vers un objet.

1.11 Polymorphisme : idée générale

Le **polymorphisme** signifie qu'une même abstraction peut prendre plusieurs formes. Le cours en présente plusieurs manifestations :

- plusieurs signatures d'une même opération ;
- plusieurs structures compatibles ;
- plusieurs implémentations d'un même service ;
- des structures génériques applicables à plusieurs types.

Le polymorphisme sera détaillé dans les chapitres sur la surcharge, la redéfinition, l'héritage et les interfaces.

1.12 Modélisation objet et UML

Un **modèle** est une représentation du système informatique que l'on veut réaliser. La modélisation procède par raffinements successifs : on part des éléments du domaine réel puis on aboutit à des concepts suffisamment précis pour être implémentés.

Le cours oppose cette démarche à la décomposition procédurale descendante et la décrit comme une construction plus **ascendante**, à partir de composants unitaires.

UML est un formalisme graphique normalisé par l'OMG. Ce n'est pas en lui-même une méthode complète de conception : il fournit des diagrammes permettant de représenter différents aspects statiques et dynamiques d'un système.

1.13 Technologie Java : langage, JVM et bytecode

Java est présenté comme un environnement comprenant :

- un langage orienté objet ;
- une machine virtuelle, la **JVM** ;
- un ensemble d'API ;
- des outils de développement.

Le principe fondamental de portabilité est le suivant :

Fichier .java -> compilation -> bytecode .class -> exécution par une JVM

Le bytecode est indépendant de l'architecture matérielle. Il est exécuté par une JVM disponible pour la plateforme cible.

Parmi les propriétés mises en avant dans le cours : portabilité, robustesse, absence de pointeurs explicites, gestion automatique de la mémoire, exceptions, chargement dynamique de classes, multithreading et grand nombre d'API.

1.14 Cycle de développement Java

Une application Java est organisée en classes et en packages. Un fichier source porte l'extension .java. Une classe publique doit normalement être placée dans un fichier portant le même nom.

Exemple :

```
Classe publique : Test
Fichier source : Test.java
Compilation    : javac Test.java
Résultat      : Test.class
Exécution     : java Test
```

Le cours mentionne également les variables d'environnement PATH, CLASSPATH et JAVA_HOME.

1.15 Variables et portée

Une variable possède un **nom** et un **type**. Le type détermine ce qu'elle peut contenir et les opérations autorisées.

Le cours distingue quatre catégories de variables selon le contexte :

- variable d'instance ;
- variable locale ;
- paramètre de méthode ;
- paramètre de gestionnaire d'exception.

La **portée** d'une variable correspond à la partie du programme dans laquelle son nom est accessible.

Le mot-clé **final** signifie qu'une variable ne peut plus être réaffectée après son initialisation.

```
final int MAX = 100;
```

1.16 Types primitifs

Les principaux types primitifs Java abordés dans le cours sont :

Type	Rôle
byte	entier signé sur 8 bits
short	entier signé sur 16 bits
int	entier signé sur 32 bits
long	entier signé sur 64 bits
float	réel en simple précision
double	réel en double précision
char	caractère Unicode
boolean	valeur logique true ou false

Une classe, un tableau ou une chaîne de caractères ne sont pas des types primitifs : les variables correspondantes contiennent des références vers des objets.

1.17 Expressions, instructions et blocs

Une **expression** combine des valeurs, variables, opérateurs et appels de méthodes et produit une valeur.

```
x + 2 * y
```

Une **instruction** représente une unité d'exécution complète, souvent terminée par ;.

Un **bloc** regroupe plusieurs instructions entre accolades.

```
if (x > 0) {
    y++;
    System.out.println(y);
}
```

1.18 Opérateurs et priorité

Le cours présente notamment :

- arithmétiques : +, -, *, /, % ;
- incrémentation : ++, -- ;
- relationnels : <, <=, >, >=, ==, != ;
- logiques : &&, ||, ! ;
- bit à bit : &, |, ^, ~ ;
- décalage : <<, >>, >>> ;
- affectation : =, +=, -=, etc. ;
- conditionnel ternaire : ? : ;
- création : new ;
- test de type : instanceof ;
- cast : (Type).

La **priorité** et l'**associativité** déterminent l'ordre d'évaluation. Exemple du cours :

```
int op = 1 - 2 - 3;
```

La soustraction est associative de gauche à droite : $(1 - 2) - 3 = -4$.

Lorsque l'expression devient ambiguë à la lecture, utiliser des parenthèses est préférable même si l'on connaît les règles de priorité.

1.19 Structures de contrôle

Décision

```
if (condition) {
    // ...
} else {
    // ...
}

switch (valeur) {
    case 1:
        break;
    default:
        break;
}
```

Boucles

```
while (condition) { }
```

```
do { } while (condition);
```

```
for (int i = 0; i < 10; i++) { }
```

Branchement

- break interrompt une boucle ou un switch ;
- continue passe à l'itération suivante ;
- return quitte une méthode ;
- throw lance une exception.

1.20 Tableaux

Un tableau est une structure de taille fixe dont tous les éléments sont du même type.

```
int[] tab;
tab = new int[10];
```

Ou directement :

```
int[] tab = {1, 3, 5, 2, 10};
```

L'indice commence à zéro :

```
int premier = tab[0];
int taille = tab.length;
```

Un tableau multidimensionnel s'écrit par exemple :

```
int[][] matrice = new int[3][4];
```

Le cours cite `System.arraycopy` pour copier une partie d'un tableau.

1.21 Chaînes de caractères String

Une chaîne peut être créée à partir d'un littéral :

```
| String s = "Bonjour";
```

Opérations importantes :

```
| int n = s.length();
| char c = s.charAt(0);
| String s2 = s + " !";
```

Les méthodes plus détaillées de `String` seront reprises dans le chapitre sur les classes utilitaires.

1.22 Méthode main

Le point d'entrée d'un programme Java est :

```
| public static void main(String[] args) {
|     // ...
| }
```

- `public` : la méthode est accessible depuis l'extérieur ;
- `static` : elle appartient à la classe ;
- `void` : elle ne renvoie pas de valeur ;
- `String[] args` : tableau contenant les arguments de ligne de commande.

Le nombre d'arguments est disponible via `args.length`.

1.23 Entrées-sorties simples

Pour écrire :

```
| System.out.println("Bonjour");
```

Pour lire un caractère, le cours utilise `System.in.read()` et la gestion d'une `IOException`.

Pour convertir une chaîne en entier :

```
| int valeur = Integer.parseInt(s);
```

Les entrées-sorties seront approfondies plus tard avec les streams et `java.io`.

A retenir du chapitre 1

Le coeur de l'OO est la combinaison **objet + classe + encapsulation + abstraction + hiérarchie + polymorphisme**. En Java, il faut en plus maîtriser le modèle d'exécution source -> bytecode -> JVM, les types, les variables, les opérateurs, les structures de contrôle, les tableaux et `String`.

2. Classes, objets et classes utilitaires

Cours : pages 87 à 129.

2.1 Définir une classe

Une classe définit un nouveau type. Son corps contient ses **membres**, notamment les attributs et les méthodes.

```
public class Point {  
    private int x;  
    private int y;  
  
    public void init(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
  
    public void affiche() {  
        System.out.println("x : " + x + ", y : " + y);  
    }  
}
```

Une déclaration de classe n'est pas terminée par un point-virgule.

2.2 Déclaration, instanciation et utilisation d'un objet

La déclaration d'une référence ne crée pas encore l'objet :

```
Point p;
```

L'objet est créé avec `new` :

```
p = new Point();
```

Les membres accessibles s'utilisent avec l'opérateur `.` :

```
p.init(3, 4);  
p.affiche();
```

Il faut bien distinguer la **référence** `p` de l'objet réellement alloué en mémoire.

2.3 Constructeurs

Un constructeur sert à initialiser un objet. Il :

- porte le même nom que la classe ;
- n'a pas de type de retour, même pas `void` ;
- peut être surchargé ;
- est appelé après `new`.

```

public class Point {
    private int x;
    private int y;

    public Point() {
        x = 0;
        y = 0;
    }

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

Utilisation :

```

Point p1 = new Point();
Point p2 = new Point(10, 20);

```

Si aucun constructeur n'est déclaré, Java fournit un constructeur par défaut sans argument. Dès qu'un constructeur est explicitement défini, ce comportement automatique n'est plus garanti de la même façon.

Les niveaux d'accès du constructeur contrôlent qui peut créer les instances :

- `public` : toute classe autorisée à voir la classe ;
- `protected` : sous-classes et classes du même package ;
- aucun modificateur : package ;
- `private` : uniquement la classe elle-même.

2.4 Attributs d'instance et attributs de classe

Un attribut d'instance est dupliqué pour chaque objet. Un attribut static est partagé.

```

public class Compteur {
    private int valeur;           // par objet
    private static int nbObjets; // partagé
}

```

Même distinction pour les méthodes :

```

public void incrementer() { ... } // méthode d'instance
public static int getNbObjets() { ... } // méthode de classe

```

Une méthode statique ne peut pas utiliser directement `this`, car elle n'est pas exécutée sur une instance particulière.

2.5 Modificateurs de visibilité et final

private

Accessible uniquement depuis la classe elle-même.

Visibilité par défaut

Sans modificateur, le membre est visible par les classes du même package.

protected

Visible dans le package et dans les sous-classes selon les règles Java d'héritage.

public

Visible partout où la classe est accessible.

static

Le membre appartient à la classe et non à une instance.

final

Une variable `final` ne peut plus être réaffectée après son initialisation. Pour une référence d'objet, cela signifie que la référence ne peut plus pointer vers un autre objet ; cela ne rend pas forcément l'objet lui-même immuable.

2.6 Signature et résultat d'une méthode

Une méthode précise :

- un niveau d'accès ;
- éventuellement des modificateurs (`static`, `final`, `abstract`...) ;
- un type de retour ou `void` ;
- un nom ;
- une liste de paramètres ;
- éventuellement une clause `throws`.

```
public static int somme(int a, int b) {
    return a + b;
}
```

Le type de l'expression retournée doit être compatible avec le type annoncé.

2.7 Surcharge de méthodes

La **surcharge** consiste à définir plusieurs méthodes de même nom avec des listes de paramètres différentes.

```
public void deplacer(int x, int y) { ... }
public void deplacer(Point p) { ... }
```

Le compilateur choisit la méthode en fonction du nombre et du type des arguments.

2.8 Passage des arguments par valeur

Le cours insiste sur un point fondamental : **Java passe les arguments par valeur**.

Type primitif

La méthode reçoit une copie de la valeur :

```
static void modifier(int x) {
    x = 100;
}
```

L'entier de l'appelant n'est pas modifié.

Référence d'objet

La méthode reçoit une copie de la référence. Elle peut donc modifier l'objet pointé, mais réaffecter le paramètre local ne change pas la référence de l'appelant.

```
static void modifier(Point p) {
    p.setX(100); // modifie l'objet
    p = new Point(); // ne remplace pas la référence de l'appelant
}
```

C'est la raison du résultat du type d'exemple présenté dans le cours : le contenu de l'objet peut changer, mais l'affectation locale du paramètre ne remplace pas la référence extérieure.

2.9 this

this désigne l'objet courant.

Il est utile lorsque le paramètre porte le même nom qu'un attribut :

```
public Point(int x, int y) {
    this.x = x;
    this.y = y;
}
```

this.x désigne l'attribut, alors que x seul désigne le paramètre local.

2.10 Composition d'objets

La **composition** représente une relation de type "possède un" ou "contient un".

```
public class Cercle {
    private Point centre;
    private float rayon;

    public Cercle(int x, int y, float rayon) {
        centre = new Point(x, y);
        this.rayon = rayon;
    }
}
```

Un Cercle contient ici un Point. Cette relation est différente de l'héritage : un cercle **possède** un point, mais un cercle **n'est pas** un point.

2.11 Classes enveloppes

Les types primitifs ont des classes enveloppes correspondantes :

Primitif	Classe enveloppe
int	Integer
double	Double
float	Float
boolean	Boolean
byte	Byte
char	Character
long	Long
short	Short

Elles sont utiles lorsqu'une API attend un objet plutôt qu'un type primitif.

Conversion classique :

```
int n = Integer.parseInt("122");
Integer obj = Integer.valueOf("122");
```

2.12 Classe Character

Le cours cite notamment :

```
char c = objet.charValue();
boolean maj = Character.isUpperCase(c);
```

Character fournit des opérations de test et de conversion autour des caractères.

2.13 Classe String

String représente une chaîne de caractères. Le cours la présente comme adaptée aux chaînes qui ne sont pas modifiées en place.

Méthodes importantes :

```
s.length();
s.charAt(i);
s.substring(debut);
s.substring(debut, fin);
s.indexOf('a');
s.lastIndexOf('a');
s.toUpperCase();
```

Exemple :

```
String nom = "nino";
String maj = nom.toUpperCase(); // "NINO"
```

2.14 Classe StringBuffer

StringBuffer permet de construire et modifier une séquence de caractères.

```
StringBuffer sb = new StringBuffer();  
sb.append("Bonjour");  
sb.insert(7, " Nino");  
sb.setCharAt(0, 'b');
```

Le cours mentionne également `capacity()` et `length()` pour distinguer espace alloué et longueur utilisée.

2.15 Conversions d'objets et de chaînes

Toute classe Java hérite indirectement de `Object`, qui fournit notamment `toString()`.

```
String texte = objet.toString();
```

Pour convertir une chaîne vers un nombre, on utilise les classes enveloppes :

```
int a = Integer.parseInt("42");  
double d = Double.parseDouble("3.14");
```

A retenir du chapitre 2

Savoir écrire une classe complète avec attributs privés, constructeurs, méthodes, `this`, `static` et niveaux de visibilité. Savoir expliquer le passage par valeur, distinguer composition et héritage, et connaître le rôle de `String`, `StringBuffer` et des classes enveloppes.

3. Héritage, généricité et polymorphisme

Cours : pages 130 à 207.

3.1 Pourquoi utiliser l'héritage ?

L'héritage permet de réutiliser et de factoriser du code existant. Lorsqu'on identifie plusieurs classes partageant des attributs ou comportements, on peut remonter ces éléments communs dans une classe plus générale.

Ses objectifs principaux sont :

- réutilisation ;
- réduction des duplications ;
- représentation plus claire de la hiérarchie des concepts ;
- maintenance facilitée ;
- spécialisation progressive de comportements existants.

L'héritage ne doit pas être utilisé uniquement pour éviter de recopier du code : la relation conceptuelle **“est une sorte de”** doit être cohérente.

3.2 Règles de l'héritage Java

Le cours donne trois règles fondamentales :

1. une classe Java n'hérite directement que d'une seule classe ;
2. sans héritage explicite, une classe descend de `Object` ;
3. les cycles d'héritage sont interdits.

La hiérarchie de classes forme donc un arbre enraciné en `Object`.

Syntaxe :

```
public class Chien extends Animal {
    // ...
}
```

3.3 Membres hérités

Une sous-classe récupère les membres héritables de la superclasse. Leur possibilité d'utilisation dépend de la visibilité.

```
public class Animal {
    public String nom;
    private int age;

    public int getAge() {
        return age;
    }
}

public class Vertebre extends Animal {
    private int nbVertebres;
}
```

Vertebre possède un état comprenant la partie héritée de `Animal` et sa partie propre. Cependant, un membre `private` n'est pas directement accessible depuis le code de la sous-classe.

3.4 Généralisation et spécialisation

Généralisation

On repère plusieurs classes avec des caractéristiques communes et on crée une superclasse qui factorise ces éléments.

Spécialisation

On part d'une classe générale et on définit une sous-classe plus précise qui ajoute ou redéfinit des propriétés et comportements.

Test conceptuel important :

D hérite de B si l'on peut dire : "D est une sorte de B".

Exemple : Cerf est une sorte d'Animal.

3.5 Vocabulaire

Termes équivalents souvent rencontrés :

- superclasse = classe mère = classe de base ;
- sous-classe = classe fille = classe dérivée ;
- une sous-classe **étend** une superclasse.

Une classe peut avoir plusieurs sous-classes et la profondeur de la hiérarchie peut être de plusieurs niveaux.

3.6 Principe de substitution

Un objet d'une sous-classe peut être utilisé partout où une instance de la superclasse est attendue, tant que l'on reste dans les opérations garanties par le type de la superclasse.

```
Cerf c = new Cerf();
Animal a = c;
```

La conversion de la sous-classe vers la superclasse est naturelle et implicite.

L'inverse nécessite davantage de précautions :

```
Animal a = new Cerf();
Cerf c = (Cerf) a;
```

Le cast explicite ne fonctionne à l'exécution que si l'objet réel est compatible avec `Cerf`.

3.7 Cast implicite et cast explicite

Upcast

Sous-classe vers superclasse : implicite.

```
Chien c = new Chien();
Animal a = c;
```

Downcast

Superclasse vers sous-classe : explicite.

```
Chien c2 = (Chien) a;
```

On peut vérifier le type avec :

```
if (a instanceof Chien) {
    Chien c2 = (Chien) a;
}
```

3.8 Héritage ou composition ?

L'héritage exprime **“est un”**. La composition exprime **“a un”** ou **“contient un”**.

```
Chien est un Animal    -> héritage
Voiture a un Moteur    -> composition
Cercle a un Point comme centre -> composition
```

Choisir la relation sémantiquement correcte évite des hiérarchies artificielles.

3.9 Recherche de méthode

Lorsqu'une méthode est appelée sur un objet, Java recherche une implémentation appropriée en tenant compte de la classe réelle et de la hiérarchie. Si la méthode n'est pas définie à ce niveau, la recherche remonte dans les superclasses.

C'est la base de la redéfinition et du polymorphisme dynamique.

3.10 Visibilité et protected

Le cours insiste sur la différence entre les niveaux de visibilité :

Visibilité	Classe	Même package	Sous-classe	Autres
private	oui	non	non directement	non
défaut	oui	oui	si package	non
protected	oui	oui	oui selon règles d'héritage	non
public	oui	oui	oui	oui

Le cours recommande de ne pas rendre trop de données `protected` simplement pour faciliter l'héritage. L'encapsulation doit rester aussi forte que possible.

3.11 Surcharge et redéfinition

Surcharge - polymorphisme statique

Même nom, signatures différentes :

```
void afficher(int x) { ... }
void afficher(String s) { ... }
```

Le choix est essentiellement résolu à la compilation.

Redéfinition - polymorphisme dynamique

Une sous-classe fournit une nouvelle implémentation d'une méthode héritée ayant la même signature.

```
class Animal {
    public void parler() {
        System.out.println("...");
    }
}

class Chien extends Animal {
    @Override
    public void parler() {
        System.out.println("Ouaf");
    }
}
```

Avec :

```
Animal a = new Chien();
a.parler();
```

la version correspondant à l'objet réel `Chien` est utilisée.

3.12 Règles de redéfinition

La méthode redéfinie doit rester compatible avec la méthode de la superclasse. Le cours met notamment en avant :

- même signature ;
- visibilité non plus restrictive ;
- possibilité d'accéder à la version de la superclasse avec `super` ;
- une méthode `final` ne peut pas être redéfinie ;
- les méthodes `static` sont masquées plutôt que redéfinies dynamiquement.

Exemple :

```
@Override
public void affiche() {
    super.affiche();
    System.out.println("Informations supplémentaires");
}
```

3.13 Constructeurs et héritage

Les constructeurs ne sont pas hérités. Chaque classe initialise sa propre partie d'état.

Le constructeur de la sous-classe doit commencer par l'appel approprié au constructeur de la superclasse, explicitement avec `super(...)` ou implicitement dans certains cas.

```

class Point {
    protected int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

class PointCouleur extends Point {
    private int couleur;

    public PointCouleur(int x, int y, int couleur) {
        super(x, y);
        this.couleur = couleur;
    }
}

```

Si la superclasse ne possède pas de constructeur sans argument, la sous-classe doit appeler explicitement un constructeur existant avec `super(...)`.

3.14 Héritage multiple

Java interdit l'héritage multiple de **classes**. Une classe ne peut donc pas écrire :

```
class C extends A, B // interdit
```

Le cours explique que l'héritage multiple de classes provoque des ambiguïtés, notamment lorsque deux superclasses définissent le même membre ou dans le cas d'une hiérarchie en losange.

Java autorise cependant l'implémentation de plusieurs **interfaces**.

3.15 Interfaces comme contrat

Une interface décrit un contrat de services. Une classe qui l'implémente promet de fournir les opérations correspondantes.

```

public interface Affichable {
    void affiche();
}

public class Point implements Affichable {
    @Override
    public void affiche() {
        System.out.println("Point");
    }
}

```

Une interface permet notamment de donner un type commun à des classes qui ne partagent pas nécessairement la même classe mère spécialisée.

3.16 Polymorphisme par héritage et interfaces

Exemple du cours :

```
interface Herbivore { }
class Animal { }
class Cerf extends Animal implements Herbivore { }
```

Le même objet peut être vu sous plusieurs types :

```
Cerf c = new Cerf();
Animal a = c;
Herbivore h = c;
Object o = c;
```

Le type de la variable détermine les opérations visibles à la compilation, tandis que le type réel de l'objet intervient pour les méthodes redéfinies.

3.17 Typage statique et type dynamique

Dans :

```
Animal a = new Chien();
```

- type statique de a : Animal ;
- type réel ou dynamique de l'objet : Chien.

Le type statique contrôle notamment ce que le compilateur autorise à appeler directement. Le type dynamique participe au choix de l'implémentation redéfinie pendant l'exécution.

3.18 Classes abstraites

Une classe abstraite représente un concept général qui ne doit pas être instancié.

```
public abstract class Forme {
    public abstract double aire();
}
```

Une sous-classe concrète doit fournir l'implémentation des méthodes abstraites :

```
public class Cercle extends Forme {
    private double rayon;

    @Override
    public double aire() {
        return Math.PI * rayon * rayon;
    }
}
```

Une classe abstraite peut contenir des attributs, des constructeurs et des méthodes déjà implémentées.

3.19 Généricité

La **généricité** permet d'écrire du code indépendant d'un type précis tout en conservant des contrôles de types.

Exemple typique :

```
List<String> noms = new ArrayList<>();
List<Integer> nombres = new ArrayList<>();
```

Le même concept `List<T>` peut être utilisé avec différents types.

L'intérêt principal est d'éviter les casts dangereux et d'exprimer plus précisément les types attendus.

Exemple de classe générique :

```
public class Boite<T> {
    private T valeur;

    public void set(T valeur) {
        this.valeur = valeur;
    }

    public T get() {
        return valeur;
    }
}
```

A retenir du chapitre 3

Le point essentiel est de distinguer :

```
Surcharge    -> même nom, paramètres différents, choix statique
Redéfinition -> même signature dans une sous-classe, comportement dynamique
Héritage     -> relation "est un"
Composition  -> relation "a un"
Upcast       -> sous-classe vers superclasse, naturel
Downcast     -> superclasse vers sous-classe, explicite et potentiellement risqué
```

4. Interfaces et expressions lambda

Cours : pages 208 à 275.

4.1 Rappels d'héritage : super et redéfinition

Le cours reprend l'exemple de `Point` et `ColoredPoint` pour illustrer :

- l'appel au constructeur de la superclasse avec `super(...)` ;
- l'appel à une méthode de la superclasse avec `super.methode()` ;
- la redéfinition d'une méthode ;
- le cast nécessaire pour retrouver une opération propre à la sous-classe lorsque la référence a un type de superclasse.

Exemple :

```
Point p = new ColoredPoint(4, 5, 1);
p.affiche();
((ColoredPoint) p).setCouleur(2);
```

4.2 Classe Object

`Object` est la racine de la hiérarchie des classes Java. Toute classe en hérite directement ou indirectement.

Le cours présente plusieurs méthodes importantes :

`equals`

Par défaut, l'égalité correspond à l'identité des références. Une classe peut redéfinir `equals` pour définir une égalité logique fondée sur son contenu.

`hashCode`

Associé à `equals`, notamment dans les structures de hachage.

`toString`

Retourne une représentation textuelle de l'objet. Très utile pour le débogage et l'affichage.

```
@Override
public String toString() {
    return "Point(" + x + ", " + y + ")";
}
```

`getClass`

Permet d'obtenir une représentation de la classe réelle de l'objet à l'exécution.

Le cours cite également `clone` et `finalize` dans sa présentation de `Object`.

4.3 Classes et méthodes finales

Une classe `final` ne peut pas être étendue :

```
public final class Configuration { }
```

Une méthode `final` ne peut pas être redéfinie :

```
public final void verifier() { }
```

L'objectif peut être lié à la conception ou à la sécurité d'un comportement qui ne doit pas changer dans les sous-classes.

4.4 Interfaces : définition

Une interface définit un protocole ou contrat de communication.

```
public interface Affichable {  
    void affiche();  
}
```

Une classe qui implémente l'interface doit fournir une implémentation des méthodes abstraites requises :

```
public class Point implements Affichable {  
    @Override  
    public void affiche() {  
        System.out.println("Point");  
    }  
}
```

Les constantes d'une interface sont implicitement `public static final`.

4.5 Méthodes default et static dans les interfaces

Le cours explique la motivation introduite avec Java 8 : faire évoluer une interface sans obliger immédiatement toutes les anciennes classes à ajouter une implémentation.

Méthode default

```
public interface Contact {  
    void envoyer(String message);  
  
    default void envoyerUrgence(String message) {  
        envoyer("URGENT: " + message);  
    }  
}
```

Une classe peut hériter de l'implémentation par défaut ou la redéfinir.

Méthode static

```
public interface Contact {  
    static int getId() {  
        return 23;  
    }  
}
```

Elle s'appelle sur l'interface :

```
int id = Contact.getId();
```

4.6 Classe abstraite ou interface ?

Le cours compare les deux concepts.

Classe abstraite	Interface
peut avoir des constructeurs	pas utilisée comme classe instanciable
peut avoir des attributs d'instance	principalement contrat de services
peut avoir plusieurs niveaux de visibilité	constantes d'interface publiques/statique/finales
peut contenir méthodes abstraites et concrètes	peut contenir méthodes abstraites, default, static
une classe n'étend qu'une classe	une classe peut implémenter plusieurs interfaces

L'interface est particulièrement adaptée à la définition d'un **rôle** ou d'une capacité commune à plusieurs classes.

4.7 Héritage multiple d'interfaces et conflit de méthodes default

Une classe peut implémenter plusieurs interfaces. Si deux interfaces fournissent des méthodes default incompatibles avec la même signature, la classe doit résoudre explicitement le conflit.

```
interface A {
    default void f() { System.out.println("A"); }
}

interface B {
    default void f() { System.out.println("B"); }
}

class C implements A, B {
    @Override
    public void f() {
        A.super.f();
    }
}
```

Le cours utilise ce mécanisme pour montrer comment Java évite l'ambiguïté du diamant d'héritage.

4.8 Motivation des expressions lambda

Les **expressions lambda** permettent de passer un traitement comme valeur de manière concise. Elles représentent des fonctions anonymes et réduisent le code nécessaire pour décrire un comportement.

Exemple classique :

```
List<String> animaux = Arrays.asList("Chat", "Chien");
animaux.forEach(System.out::println);
```

Au lieu d'écrire une boucle complète ou une classe anonyme, on fournit directement le traitement à appliquer.

4.9 Interface fonctionnelle

Une lambda correspond à une interface ne possédant qu'une seule méthode abstraite. On parle d'**interface fonctionnelle**.

```
@FunctionalInterface
interface Operation {
    long appliquer(int a, int b);
}
```

On peut alors écrire :

```
Operation addition = (a, b) -> a + b;
long r = addition.appliquer(2, 3);
```

Le compilateur infère le lien entre la lambda et l'interface fonctionnelle attendue.

4.10 Syntaxe des lambdas

Forme générale :

```
(parametres) -> expression
(parametres) -> { instructions; }
```

Exemples :

```
() -> System.out.println("run")

x -> x * 2

(a, b) -> a + b

(a, b) -> {
    int r = a + b;
    return r;
}
```

Règles importantes du cours :

- zéro, un ou plusieurs paramètres ;
- types explicites ou inférés ;
- parenthèses optionnelles pour un seul paramètre dont le type est inféré ;
- accolades facultatives pour une seule expression ;
- pour un bloc de plusieurs instructions, accolades obligatoires et return explicite si une valeur doit être renvoyée.

4.11 Interfaces fonctionnelles de java.util.function

Le cours présente quatre familles fondamentales.

Function<T, R>

Transforme une valeur et retourne un résultat.

```
Function<String, Integer> longueur = s -> s.length();
int n = longueur.apply("Java");
```

Predicate<T>

Teste une condition et renvoie un booléen.

```
Predicate<Integer> positif = x -> x > 0;
boolean b = positif.test(5);
```

Supplier<T>

Produit une valeur sans argument.

```
Supplier<String> fournisseur = () -> "bonjour";
String s = fournisseur.get();
```

Consumer<T>

Consomme une valeur et ne renvoie rien.

```
Consumer<String> afficher = s -> System.out.println(s);
afficher.accept("Hello");
```

Le cours met en avant que Consumer ne retourne aucune valeur. Une expression comme $v \rightarrow v * 3$ n'est donc pas adaptée à un Consumer<Integer> si son but est de renvoyer le résultat du calcul.

4.12 Composition de Consumer avec andThen

```
Consumer<String> c = System.out::print;
c.andThen(c).andThen(c).accept("salut-");
```

Le traitement est exécuté plusieurs fois en séquence.

4.13 Références de méthodes

Une référence de méthode est une syntaxe compacte lorsqu'une lambda ne fait qu'appeler une méthode existante.

Méthode statique

```
Math::pow
```

équivalent conceptuellement à :

```
(x, y) -> Math.pow(x, y)
```

Méthode sur une instance donnée

```
System.out::println
```

équivalent à :

```
x -> System.out.println(x)
```

Méthode d'un objet arbitraire d'un type

```
String::compareTo
```

peut correspondre à :

```
(x, y) -> x.compareTo(y)
```

Référence de constructeur

```
Personne::new
```

équivalent à une lambda créant une nouvelle instance si l'interface fonctionnelle attendue est compatible.

4.14 Lambda et threads

Le cours montre trois écritures équivalentes pour fournir un `Runnable` : classe anonyme, lambda et référence de méthode.

```
| new Thread(() -> executer()).start();
```

ou :

```
| new Thread(MaClasse::executer).start();
```

C'est un exemple important du lien entre interface fonctionnelle et lambda : `Runnable` possède une unique méthode abstraite `run()`.

A retenir du chapitre 4

Il faut maîtriser la différence entre **classe abstraite** et **interface**, comprendre les méthodes **default**, savoir reconnaître une interface fonctionnelle, écrire une lambda simple et convertir une lambda en référence de méthode lorsque cela rend le code plus lisible.

5. Packages et exceptions

Cours : pages 276 à 341.

5.1 Packages

Un **package** regroupe des classes et interfaces liées. Il sert à :

- organiser le code ;
- éviter les collisions de noms ;
- contrôler certaines visibilité ;
- structurer l'espace de noms.

Déclaration en tête de fichier :

```
package mon.projet.modele;
```

La structure des répertoires suit normalement celle du package :

```
mon/projet/modele/Point.java
```

5.2 Importer des classes

On peut utiliser le nom complet :

```
java.util.ArrayList<String> liste = new java.util.ArrayList<>();
```

ou importer la classe :

```
import java.util.ArrayList;
```

Puis :

```
ArrayList<String> liste = new ArrayList<>();
```

On peut importer plusieurs types d'un package avec * :

```
import java.util.*;
```

Le package java.lang contient des classes fondamentales et est disponible automatiquement.

5.3 Sources, classes compilées et packages

Le nom du package se reflète dans l'arborescence des sources et des .class.

Exemple :

```
Nom complet : mon.projet.Point
Source      : mon/projet/Point.java
Classe     : mon/projet/Point.class
```

Cette relation est importante lorsque l'on compile et exécute sans IDE.

5.4 Pourquoi les exceptions ?

Une **exception** est un événement anormal qui interrompt le déroulement normal des instructions.

Au lieu de mélanger le traitement normal et la gestion de toutes les erreurs, Java permet de :

- signaler une erreur avec un objet `Exception` ;
- propager l'erreur dans la pile d'appels ;
- gérer différents types d'erreurs avec différents blocs `catch` ;
- séparer logique métier et gestion d'erreur.

Exemple typique : division par zéro :

```
int i = 3;
int j = 0;
System.out.println(i / j);
```

Une `ArithmeticException` est levée à l'exécution.

5.5 throw et throws

throw

`throw` lance concrètement une exception :

```
if (x < 0) {
    throw new IllegalArgumentException("x negatif");
}
```

throws

`throws` indique dans la signature qu'une méthode peut laisser remonter une exception :

```
public void lire() throws IOException {
    // ...
}
```

Le mot-clé `throws` délègue donc la responsabilité à la méthode appelante.

5.6 Exemple d'exception contrôlée personnalisée

```
class ConstErr extends Exception {
}

class Point {
    private int x;
    private int y;

    Point(int x, int y) throws ConstErr {
        if (x < 0 || y < 0) {
            throw new ConstErr();
        }
        this.x = x;
        this.y = y;
    }
}
```

La méthode appelante doit alors traiter ou propager l'exception.

5.7 Bloc try / catch

```
try {
    Point p = new Point(-3, 5);
} catch (ConstErr e) {
    System.out.println("Erreur a la construction");
}
```

Lorsqu'une exception est levée :

1. l'exécution du bloc courant est interrompue ;
2. Java recherche un gestionnaire catch compatible ;
3. la recherche peut remonter la pile d'appels ;
4. si un gestionnaire est trouvé, il est exécuté ;
5. l'exécution continue ensuite selon la structure du programme.

5.8 Plusieurs blocs catch

Un même try peut être suivi de plusieurs gestionnaires :

```
try {
    // ...
} catch (FileNotFoundException e) {
    // cas précis
} catch (IOException e) {
    // cas plus general
}
```

L'ordre doit aller du plus spécifique vers le plus général.

5.9 Bloc finally

finally permet d'exécuter un traitement de nettoyage associé à un try, qu'une exception ait été levée ou non.

```
try {
    // traitement
} catch (Exception e) {
    // gestion
} finally {
    // nettoyage
}
```

Le cours le relie notamment à la fermeture de ressources.

5.10 Try-with-resources et AutoCloseable

Java permet de déclarer une ressource dans l'en-tête du try. Elle sera fermée automatiquement si elle implémente AutoCloseable.

```
try (PrintWriter writer = new PrintWriter(new File("test.txt"))) {
    writer.println("Hello World");
} catch (Exception e) {
    e.printStackTrace();
}
```

Cette forme évite de répéter la fermeture dans un `finally` et limite les oublis.

5.11 Classe Exception et message d'erreur

Une exception peut transporter un message :

```
throw new Exception("Valeur incorrecte");
```

Récupération :

```
catch (Exception e) {
    System.out.println(e.getMessage());
}
```

Dans une exception personnalisée :

```
public class SaisieErroneeException extends Exception {
    public SaisieErroneeException() {
        super();
    }

    public SaisieErroneeException(String message) {
        super(message);
    }
}
```

5.12 Hiérarchie des erreurs

La hiérarchie présentée dans le cours repose sur `Throwable`.

```
Object
-> Throwable
    -> Error
    -> Exception
        -> RuntimeException
```

Error

Représente des problèmes graves du système d'exécution. Ils ne sont normalement pas gérés comme des erreurs métier ordinaires.

Exception

Regroupe les exceptions gérables par l'application.

RuntimeException

Le compilateur n'oblige pas à les déclarer ou les intercepter. Exemples classiques : `NullPointerException`, erreur d'indice, etc.

5.13 Exceptions contrôlées et non contrôlées

Checked exceptions

Le compilateur impose qu'elles soient :

- soit traitées avec `try/catch` ;
- soit déclarées avec `throws`.

Unchecked exceptions

Les exceptions héritant de `RuntimeException` et les erreurs héritant de `Error` n'ont pas besoin d'être déclarées dans la signature.

5.14 Exception personnalisée complète

```
public class SaisieErroneeException extends Exception {
    public SaisieErroneeException(String message) {
        super(message);
    }
}

public class Validation {
    public static void controle(String chaine)
        throws SaisieErroneeException {
        if (chaine.isEmpty()) {
            throw new SaisieErroneeException("chaine vide");
        }
    }
}
```

Appel :

```
try {
    Validation.controle("");
} catch (SaisieErroneeException e) {
    System.out.println(e.getMessage());
}
```

A retenir du chapitre 5

La chaîne logique est : **une erreur est détectée -> throw crée/émet l'exception -> elle est traitée par catch ou déclarée avec throws -> finally ou try-with-resources assure le nettoyage.**

6. Collections et entrées-sorties

Cours : pages 342 à 376.

6.1 Framework des collections

Une collection regroupe plusieurs éléments dans une même structure. Le framework Java Collections fournit :

- des **interfaces** décrivant les contrats ;
- des **implémentations** concrètes ;
- des **algorithmes** réutilisables, souvent dans la classe Collections.

Les collections sont principalement dans `java.util`.

6.2 Interfaces fondamentales

Collection

Racine d'une grande partie de la hiérarchie. Opérations typiques :

```
size()
isEmpty()
contains(...)
add(...)
remove(...)
iterator()
clear()
toArray()
```

Set

Ensemble sans doublons.

List

Séquence ordonnée, doublons autorisés.

Map

Associe des clés à des valeurs. Une clé ne doit pas être dupliquée.

SortedSet

Ensemble maintenant un ordre.

SortedMap

Map ordonnée selon les clés.

Map n'est pas une sous-interface de Collection, même si elle appartient au même framework de collections.

6.3 Implémentations citées dans le cours

Ensembles

- HashSet : accès généralement rapide, pas d'ordre de tri garanti ;

- TreeSet : maintient un ordre.

Listes

- ArrayList : tableau redimensionnable ;
- LinkedList : structure chaînée ;
- Vector : autre structure dynamique citée dans le cours.

Maps

- HashMap ;
- TreeMap.

6.4 Itérateurs

Un Iterator permet de parcourir une collection indépendamment de sa structure interne.

```
Iterator<String> it = liste.iterator();
while (it.hasNext()) {
    String s = it.next();
    System.out.println(s);
}
```

Interface conceptuelle du cours :

```
boolean hasNext();
Object next();
void remove();
```

6.5 Tri et Comparable

Le cours cite :

```
Collections.sort(liste);
```

Pour qu'un objet soit naturellement comparable, sa classe peut implémenter Comparable.

```
class Etudiant implements Comparable<Etudiant> {
    private int note;

    @Override
    public int compareTo(Etudiant autre) {
        return Integer.compare(note, autre.note);
    }
}
```

Ainsi, une liste d'étudiants peut être triée selon l'ordre naturel défini par compareTo.

6.6 ArrayList

Une ArrayList représente une liste dynamique.

Exemple moderne et conceptuellement conforme au cours :

```

ArrayList<String> noms = new ArrayList<>();
noms.add("Alice");
noms.add("Bob");

String premier = noms.get(0);
int taille = noms.size();
noms.remove(1);

```

Le cours insiste sur le contraste avec un tableau classique : la taille logique d'une liste peut évoluer.

6.7 Notion de flot ou stream d'entrée-sortie

Un **flot** est un canal séquentiel entre le programme et une source ou destination de données.

Cycle classique :

```
ouvrir -> lire/écrire -> fermer
```

Un flot peut être connecté à :

- un fichier ;
- la mémoire ;
- une socket ;
- un canal entre traitements.

6.8 Flots d'octets et flots de caractères

Le package `java.io` distingue deux grandes familles :

Octets

- `InputStream` ;
- `OutputStream`.

Adaptés aux données binaires.

Caractères

- `Reader` ;
- `Writer`.

Adaptés aux données textuelles.

Ces classes de base sont abstraites et fournissent les opérations communes de lecture, écriture et fermeture.

6.9 Flots d'octets

Exemples de classes présentées :

```

FileInputStream / FileOutputStream
BufferedInputStream / BufferedOutputStream
DataInputStream / DataOutputStream
ObjectInputStream / ObjectOutputStream
PrintStream

```

Exemple d'écriture binaire

```
FileOutputStream fos = new FileOutputStream("data.bin");
BufferedOutputStream bos = new BufferedOutputStream(fos);
DataOutputStream dos = new DataOutputStream(bos);

dos.writeInt(2);
dos.close();
```

Le tampon réduit le nombre d'accès physiques à la ressource et peut améliorer l'efficacité.

6.10 PrintStream

System.out est un PrintStream.

On peut aussi créer un flux d'affichage vers un fichier :

```
PrintStream p = new PrintStream(new FileOutputStream("sortie.txt"));
p.println("Bonjour");
p.close();
```

6.11 Exceptions d'entrée-sortie

Les opérations d'E/S peuvent lever des exceptions, par exemple :

- IOException ;
- EOFException ;
- FileNotFoundException ;
- ObjectStreamException.

Une méthode doit donc gérer ou propager les exceptions contrôlées concernées.

6.12 Flots de caractères

Exemples :

```
FileReader / FileWriter
BufferedReader / BufferedWriter
InputStreamReader / OutputStreamWriter
PrintWriter
```

BufferedReader est notamment utile pour lire une ligne :

```
BufferedReader in = new BufferedReader(
    new InputStreamReader(System.in));

String ligne = in.readLine();
```

6.13 Conversion octets <-> caractères

InputStreamReader

Convertit les octets d'un InputStream en caractères.

OutputStreamWriter

Convertit les caractères en octets envoyés vers un `OutputStream`.

Exemple :

```
BufferedReader in = new BufferedReader(  
    new InputStreamReader(System.in));  
int n = Integer.parseInt(in.readLine());
```

6.14 Autres classes de java.io

StreamTokenizer

Découpe un flot en unités lexicales ou tokens.

File

Représente des informations de chemin et permet d'interroger ou manipuler le système de fichiers : existence, répertoire, liste, création, suppression, renommage, etc.

RandomAccessFile

Permet un accès non séquentiel dans un fichier. La méthode `seek` déplace la position de lecture/écriture.

A retenir du chapitre 6

Collections : savoir choisir entre `List`, `Set` et `Map`, utiliser un itérateur et comprendre le rôle de `Comparable`.

Entrées-sorties : savoir distinguer **octets** et **caractères**, comprendre la décoration des streams (`File` -> `Buffered` -> `Data`) et toujours penser à la fermeture des ressources.

7. Sérialisation, JDBC, threads et sockets

Cours : pages 377 à 419.

7.1 Sérialisation

La **sérialisation** transforme l'état d'un objet en une représentation permettant de le sauvegarder puis de le reconstruire plus tard.

Le cours utilise `ObjectOutputStream` et `ObjectInputStream`.

Ecriture

```
ObjectOutputStream out = new ObjectOutputStream(
    new FileOutputStream("objet.bin"));
out.writeObject(objet);
out.close();
```

Lecture

```
ObjectInputStream in = new ObjectInputStream(
    new FileInputStream("objet.bin"));
MaClasse objet = (MaClasse) in.readObject();
in.close();
```

La classe doit implémenter `Serializable` :

```
class MaClasse implements Serializable {
    // ...
}
```

`Serializable` est une interface marqueur : elle ne demande pas de méthode particulière dans le modèle présenté par le cours.

7.2 JDBC : accès aux bases relationnelles

JDBC (*Java DataBase Connectivity*) est l'API Java pour interagir avec une base de données relationnelle.

Le principe général est :

```
Programme Java -> API JDBC -> pilote JDBC -> SGBD
```

Le code JDBC reste largement indépendant du SGBD, tandis qu'un pilote adapté relie l'API au moteur choisi : PostgreSQL, MySQL, Oracle, SQLite, etc.

7.3 Connexion JDBC

Exemple de structure inspirée du cours :

```
String url = "jdbc:postgresql://localhost:5432/maBase";
Properties props = new Properties();
props.setProperty("user", "john");
props.setProperty("password", "doe");

Connection connection = DriverManager.getConnection(url, props);
```

Les ressources de base de données doivent être fermées. La forme try-with-resources est adaptée :

```
try (Connection c = DriverManager.getConnection(url, props)) {
    // requetes
}
```

7.4 Statement, PreparedStatement et ResultSet

Requête simple

```
Statement stmt = connection.createStatement();
stmt.executeUpdate("CREATE TABLE ...");
stmt.close();
```

Requête paramétrée

```
String sql = "INSERT INTO product(name, price) VALUES(?, ?)";

PreparedStatement ps = connection.prepareStatement(
    sql, Statement.RETURN_GENERATED_KEYS);

ps.setString(1, product.getName());
ps.setDouble(2, product.getPrice());
ps.executeUpdate();
```

Clés générées

```
ResultSet rs = ps.getGeneratedKeys();
if (rs.next()) {
    int id = rs.getInt(1);
}
```

Le point important est de comprendre la succession **connexion -> requête -> résultat -> fermeture des ressources**.

7.5 Threads : définition

Un **thread** est un flot d'exécution à l'intérieur d'un processus. Un programme peut ainsi exécuter plusieurs tâches de manière concurrente.

Le multithreading peut améliorer la réactivité et l'utilisation des ressources, mais introduit des problèmes lorsque plusieurs threads accèdent aux mêmes données.

7.6 Créer un thread en étendant Thread

```
class MonThread extends Thread {
    @Override
    public void run() {
        System.out.println("Execution");
    }
}

MonThread t = new MonThread();
t.start();
```

Il faut appeler `start()` pour demander la création d'un nouveau flot d'exécution. Appeler directement `run()` n'est qu'un appel de méthode ordinaire.

7.7 Créer un thread avec Runnable

```
class MaTache implements Runnable {
    @Override
    public void run() {
        System.out.println("Execution");
    }
}

Thread t = new Thread(new MaTache());
t.start();
```

Avec une lambda :

```
Thread t = new Thread(() -> System.out.println("Execution"));
t.start();
```

Cette forme montre l'intérêt de `Runnable` comme interface fonctionnelle.

7.8 Méthodes importantes des threads

Le cours présente notamment :

`start()`

Démarre le thread et provoque l'exécution de `run()` dans un nouveau flot.

`run()`

Contient le travail du thread.

`join()`

Attend la fin d'un autre thread.

```
t1.start();
t2.start();
t1.join();
t2.join();
```

sleep(...)

Suspend temporairement le thread courant.

yield()

Suggère au planificateur de laisser la main à d'autres threads.

Le cours évoque également les mécanismes wait, notify et notifyAll pour coordonner des threads sans polling permanent.

7.9 Problème de concurrence sur une donnée partagée

Supposons deux threads qui exécutent `compteur++` 5000 fois. On pourrait attendre 10000, mais l'incréméntation n'est pas nécessairement une opération atomique : les deux threads peuvent lire et écrire des valeurs en concurrence.

```
class Compteur {
    int valeur = 0;

    void incrementer() {
        valeur++;
    }
}
```

Avec deux threads, le résultat peut être inférieur à 10000 selon les entrelacements d'exécution.

7.10 synchronized

Le mot-clé `synchronized` protège une section critique avec un verrou intrinsèque.

Méthode synchronisée

```
synchronized void incrementer() {
    valeur++;
}
```

Un seul thread à la fois peut exécuter cette méthode synchronisée sur le même objet.

Bloc synchronisé

```
void incrementer() {
    synchronized (this) {
        valeur++;
    }
}
```

Le bloc limite la zone protégée à la partie réellement critique.

7.11 Verrous et remarques sur synchronized

Le cours rappelle :

- une méthode synchronisée d'instance verrouille l'objet ;
- une méthode synchronisée statique verrouille au niveau de la classe ;
- le verrou est libéré à la sortie de la section synchronisée ;
- `synchronized` s'applique aux méthodes ou blocs, pas aux variables ;
- synchroniser une section trop grande réduit le parallélisme ;
- `wait`, `notify` et `notifyAll` participent à la coordination autour d'un moniteur.

7.12 Problème de visibilité mémoire

Même lorsqu'il n'y a pas besoin de verrouillage complexe, un thread peut ne pas voir immédiatement une modification effectuée par un autre thread.

Exemple conceptuel :

```
| private boolean running = true;
```

Un thread boucle sur `running`, un autre passe la variable à `false`. Sans mécanisme de visibilité, le premier peut continuer à utiliser une valeur mise en cache.

7.13 volatile

Le mot-clé `volatile` garantit que les lectures et écritures de la variable utilisent une visibilité cohérente entre threads.

```
| private volatile boolean running = true;
```

Le cours recommande notamment `volatile` pour :

- drapeaux booléens ;
- variables d'état ;
- cas où un thread écrit et d'autres lisent ;
- situations sans opération composée nécessitant l'atomicité.

7.14 volatile ou synchronized ?

Propriété	volatile	synchronized
Visibilité entre threads	oui	oui
Atomicité d'une section critique	non	oui
Verrou	non	oui
Coût	léger	plus élevé
Cas typique	drapeau d'état	mise à jour de ressource partagée

Une opération composée comme `compteur++` nécessite plus qu'une simple visibilité : elle implique lecture, calcul et écriture. `volatile` seul ne la rend pas atomique.

7.15 Communication TCP et sockets

Dans le modèle client-serveur :

- le **serveur** fournit un service ;
- le **client** demande le service ;
- l'adresse IP identifie la machine ;
- le port identifie un point de service ;
- une **socket** représente une extrémité de communication.

TCP est orienté connexion : une connexion est établie avant l'échange de données.

7.16 Serveur TCP avec ServerSocket

Etapes conceptuelles du cours :

1. créer une ServerSocket ;
2. l'associer à une adresse/port avec bind ;
3. attendre une connexion avec accept() ;
4. récupérer une Socket de dialogue ;
5. lire et écrire sur les flux ;
6. fermer la socket de dialogue.

Exemple :

```
ServerSocket serveur = new ServerSocket();
serveur.bind(new InetSocketAddress(5000));

Socket client = serveur.accept();
// dialogue
client.close();
serveur.close();
```

accept() est bloquant : le thread attend tant qu'aucun client ne se connecte.

7.17 Client TCP

Etapes :

```
Socket socket = new Socket();
socket.connect(new InetSocketAddress("127.0.0.1", 5000));
```

Puis le client communique avec le serveur par les flux de la socket.

7.18 Flux d'une socket

Une connexion TCP fournit deux directions :

```
InputStream in = socket.getInputStream();
OutputStream out = socket.getOutputStream();
```

Le client et le serveur peuvent ainsi produire et consommer des données dans les deux sens.

On peut envelopper ces flux dans des classes plus pratiques, comme pour n'importe quel stream Java :

```
BufferedReader in = new BufferedReader(
    new InputStreamReader(socket.getInputStream()));

PrintWriter out = new PrintWriter(
    socket.getOutputStream(), true);
```

7.19 Serveur itératif

Un serveur itératif traite un seul client à la fois :

```
while (true) {
    Socket s = serveur.accept();
    traiterClient(s);
    s.close();
}
```

Le prochain client doit attendre la fin du précédent.

7.20 Serveur concurrent

Un serveur concurrent crée un nouveau thread pour chaque connexion.

```
while (true) {
    Socket s = serveur.accept();
    new Thread(() -> traiterClient(s)).start();
}
```

Le thread principal reste disponible pour accepter de nouveaux clients, tandis que des threads exécutants gèrent les dialogues en parallèle.

Le cours distingue donc :

```
Thread veilleur -> attend sur accept()
Thread executant -> dialogue avec un client
```

7.21 Suivi des sockets

Le cours cite des commandes système permettant d'observer les connexions TCP :

```
netstat -t
ss -t
```

et plusieurs options permettant notamment d'afficher les sockets en écoute, les processus associés ou de désactiver la résolution des noms.

A retenir du chapitre 7

- Sérialisation : objet <-> flux d'octets avec `ObjectOutputStream` / `ObjectInputStream` et `Serializable`.
- JDBC : `Connection` -> requête (`Statement`/`PreparedStatement`) -> `ResultSet` -> fermeture.
- Threads : `start()` crée le nouveau flot, `run()` contient le travail, `join()` attend.
- Concurrency : `synchronized` protège l'atomicité d'une section critique ; `volatile` traite principalement la visibilité.

- Sockets : `ServerSocket.accept()` côté serveur, `Socket.connect()` côté client, puis échange via `InputStream` et `OutputStream`.

8. Fiche de révision finale

8.1 Différences à connaître absolument

Classe vs objet

Classe : modèle / type
Objet : instance concrète de la classe

Attribut d'instance vs static

Instance : une valeur par objet
static : une valeur commune à la classe

Héritage vs composition

Héritage : "est un"
Composition : "a un"

Surcharge vs redéfinition

Surcharge : même nom, paramètres différents
Redéfinition : même signature dans une sous-classe

Classe abstraite vs interface

Classe abstraite : base partiellement implémentée, attributs et constructeurs possibles
Interface : contrat de services, implémentations multiples possibles

throw vs throws

throw : lance une exception
throws : annonce qu'une méthode peut propager une exception

Checked vs unchecked

Checked : traitement ou déclaration obligatoire
Unchecked : RuntimeException / Error, pas d'obligation de déclaration

List vs Set vs Map

List : ordonnée, doublons possibles
Set : pas de doublons
Map : association clé -> valeur

InputStream vs Reader

InputStream : octets
Reader : caractères

synchronized vs volatile

synchronized : visibilité + exclusion mutuelle / atomicité de la section critique
volatile : visibilité sans verrouillage ni atomicité des opérations composées

start() vs run()

start() : crée/démarre un nouveau thread puis provoque run()
run() : appelé directement, reste un appel de méthode ordinaire

8.2 Pièges classiques

1. Confondre une référence avec l'objet : `Point p`; ne crée pas encore de `Point`.
2. Oublier que Java passe les arguments par valeur, y compris les références.
3. Croire qu'une méthode `static` peut utiliser directement `this`.
4. Confondre surcharge et redéfinition.
5. Faire un `downcast` sans vérifier le type réel.
6. Utiliser l'héritage pour une relation qui devrait être une composition.
7. Oublier `super(...)` lorsque la superclasse ne fournit pas le constructeur attendu.
8. Oublier d'implémenter toutes les méthodes abstraites d'une interface ou d'une classe abstraite.
9. Donner à une `lambda` un corps incompatible avec la méthode de l'interface fonctionnelle.
10. Oublier de traiter ou propager une exception contrôlée.
11. Oublier de fermer fichiers, flux, connexions ou statements.
12. Penser que `volatile` rend `compteur++` atomique.
13. Appeler `run()` au lieu de `start()` en pensant créer un thread.
14. Traiter tous les clients dans le thread principal d'un serveur si l'on veut un comportement concurrent.

8.3 Mini-exemple récapitulatif OO

```

interface Affichable {
    void affiche();
}

abstract class Forme implements Affichable {
    private String nom;

    public Forme(String nom) {
        this.nom = nom;
    }

    public String getNom() {
        return nom;
    }

    public abstract double aire();
}

class Cercle extends Forme {
    private double rayon;

    public Cercle(String nom, double rayon) {
        super(nom);
        this.rayon = rayon;
    }

    @Override
    public double aire() {
        return Math.PI * rayon * rayon;
    }

    @Override
    public void affiche() {
        System.out.println(getNom() + " aire=" + aire());
    }
}

```

Utilisation polymorphique :

```

Forme f = new Cercle("C1", 2.0);
f.affiche();

```

Cet exemple rassemble : abstraction, encapsulation, héritage, classe abstraite, interface, constructeur, **super**, redéfinition et polymorphisme.

8.4 Questions types à savoir traiter

“Qu’est-ce que l’encapsulation ?”

Réponse attendue : cacher l’état et l’implémentation internes, exposer une interface de services, utiliser la visibilité pour protéger l’intégrité et permettre l’évolution de l’implémentation.

“Pourquoi utiliser l’héritage ?”

Réponse attendue : factorisation, réutilisation, hiérarchie de concepts, spécialisation ; seulement si la relation “est une sorte de” est cohérente.

“Que se passe-t-il dans `Animal a = new Chien();` ?”

`a` a pour type statique `Animal`, l’objet réel est un `Chien`. Les opérations directement accessibles dépendent du type statique ; la redéfinition de méthodes peut être résolue dynamiquement sur le type réel.

“Pourquoi `p = new Point()` dans une méthode ne remplace-t-il pas la référence de l’appelant ?”

Parce que la méthode reçoit une copie de la référence. Réaffecter la copie ne modifie pas la variable de l’appelant. En revanche, modifier l’objet pointé à travers cette copie peut modifier l’état visible par l’appelant.

“Différence entre interface fonctionnelle et lambda ?”

L’interface fonctionnelle définit le type et la signature du comportement. La lambda fournit une implémentation concise de son unique méthode abstraite.

“Que fait un `catch` ?”

Il intercepte une exception compatible levée dans le `try` ou propagée depuis une méthode appelée, puis exécute le traitement de gestion associé.

“Pourquoi `PreparedStatement` ?”

Il permet de préparer une requête paramétrée et de fournir séparément les valeurs avec `setString`, `setDouble`, etc. Le cours l’utilise pour les insertions JDBC.

“Pourquoi un résultat de compteur peut être faux avec deux threads ?”

Parce que l’incréméntation est une opération composée pouvant être entrelacée entre threads. Il faut protéger la section critique, par exemple avec `synchronized`.

“Que fait `accept()` ?”

Côté serveur TCP, `accept()` attend une demande de connexion et renvoie une `Socket` permettant de dialoguer avec le client connecté.

8.5 Plan de révision conseillé

Pour une première passe, mémoriser les définitions et différences de la fiche finale. Pour une deuxième passe, réécrire de mémoire un exemple pour chaque notion : classe, constructeur, héritage, interface, lambda, exception, collection, fichier, thread et socket. Pour une troisième passe, prendre des extraits de code et prévoir le résultat ou identifier l’erreur de compilation.

Conclusion

Le fil directeur du cours est le suivant : on part de la représentation d'un système avec des objets encapsulés, on apprend à écrire ces objets en Java, puis à les organiser par héritage et interfaces. On ajoute ensuite des mécanismes permettant de rendre le programme plus générique et robuste : lambda, packages, exceptions et collections. Enfin, le cours ouvre vers la persistance, les bases de données, la concurrence et les communications réseau.

Une bonne maîtrise ne consiste pas seulement à connaître la syntaxe Java. Il faut surtout comprendre **quel concept de modélisation chaque construction Java représente** et savoir expliquer les conséquences sur l'organisation, la sécurité, la réutilisation et l'évolution du programme.